GNNを用いたソースコード変更差分に対する不正機能検出手法の提案と評価

山本 直生[1,a)]　葛野 弘樹[1]　矢野 智彦[1]　瀧田 愼[1]　白石 善明[1]

概要：IoT 機器を制御するファームウェアには情報窃取や無断通信，任意コード実行など攻撃者が不正に利用可能な特定の機能を持つ脆弱性が報告されている．攻撃者が不正に利用可能な脆弱性を不正機能と定義する．既存研究では特定の種類の不正機能に特化しており，プログラムの依存関係を考慮できないため，特定の入力条件やコマンド実行により発動する不正機能に対して検出が困難な課題が存在する．本研究では，グラフニューラルネットワーク（GNN）を用いた不正機能検出手法を提案する．提案手法では，プログラムの制御依存，データ依存関係を表現するプログラム依存グラフを用い，GNN により文字列や命令の一致のみでなく依存関係を学習し不正機能の予測を行う．GNN にて複数の文を横断する依存関係を学習し，既存研究では検出が困難な不正機能を検出可能とする．検出対象はファームウェアの更新時に挿入されるソースコードに含まれる不正機能とし，更新前後のソースコードの変更差分に対して不正機能を含むか否かを 2 値分類する．評価では，ROSARUM を不正機能データセットとし，正常更新を含むソースコードに対して検出評価を実施した．ファームウェアの更新による不正機能の挿入に対する検出精度を評価し，提案手法が F1 スコア 91.43%を示すこと，他の脆弱性検出手法と比較して高い分類精度を示すことを確認した．

キーワード：不正機能，グラフニューラルネットワーク，プログラム依存グラフ

Proposal and Evaluation of a Malicious Function Detection Method for Source Code Changes Using GNN

Naoki Yamamoto[1,a)]　Hiroki Kuzuno[1]　Tomohiko Yano[1]　Makoto Takita[1]　Yoshiaki Shiraishi[1]

Abstract: Vulnerabilities with specific functions that attackers can exploit illicitly, such as information theft, unauthorized communication, and arbitrary code execution, have been reported in firmware that controls IoT devices. We define vulnerabilities that can be illicitly exploited by attackers as malicious functions. Existing studies specialize in specific types of malicious functions and cannot consider program dependency relationships, presenting a challenge in making it difficult to detect malicious functions triggered by specific input conditions or command execution. In this study, we propose a malicious function detection method using Graph Neural Network (GNN). The proposed method uses program dependency graphs, which represent the control and data dependency relationships of a program, and employs GNN to learn dependency relationships rather than merely matching strings and instructions, thereby predicting malicious functions. By using GNN to learn dependency relationships spanning multiple statements, the proposed method enables the detection of malicious functions that are difficult to detect with existing studies. The detection target is malicious functions contained in source code inserted during firmware updates, and the method performs binary classification on the change diff of the source code before and after the update to determine whether it contains a malicious function. In the evaluation, ROSARUM was used as the malicious function dataset, and detection evaluation was conducted on source code including normal updates. We evaluated the detection accuracy against the insertion of malicious functions through firmware updates and confirmed that the proposed method achieves an F1-score of 91.43% and demonstrates higher classification accuracy compared to other vulnerability detection methods.

Keywords: malicious functionality, graph neural network, program dependence graph

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

IoT（Internet of Things）機器は日常に広く普及しており，多くはファームウェアにより制御される．IoT 機器中のファームウェアやオープンソースソフトウェア（OSS）内には設計上のミスや実装上の不備に起因する脆弱性が存在する場合がある．本研究では脆弱性のうち，情報窃取や無断通信，任意コード実行など，「攻撃者が不正に利用可能な特定の機能を持つ脆弱性」を不正機能と定義する．

不正機能の検出手法として，ファームウェアを対象に特定の静的データに依存した不正機能を検出する手法 [1] や隠された Web インターフェースを特定する手法 [2] が提案されている．しかし，特定の種類の不正機能検出に特化しており，様々な種類の不正機能を検出するのが難しい．また，脆弱性検出手法では機械学習を用いて既知の脆弱性を 2 値分類する手法 [3], [4] が提案されているが，不正機能検出に適用する場合，不正機能に関する依存関係を抽出することが難しく，各手法に対して以下の課題が存在する．

- 文字列などの静的データや隠し Web インタフェースに起因する不正機能など特定の不正機能に特化しており [1], [2]，実行時の状態に依存して発動する不正機能など，特定の入力条件やコマンド実行により発動する不正機能に対して検出が困難である．
- 機械学習を用いた脆弱性検出手法 [3], [4] は，主に文単位または関数単位で既知の脆弱性に共通する特徴を検出する．一方で，不正機能は複数の文を横断して実装される場合があり，各行や各関数を個別に分類する手法では処理の依存関係を考慮した検出が困難である．

本研究では，グラフニューラルネットワーク（Graph Neural Network，GNN）を用いた不正機能検出手法を提案する．提案手法では，更新時に挿入される不正機能を検出対象とする．

ソースコードを解析対象とし，ファイル単位で制御依存，データ依存関係を表現したプログラム依存グラフ（Program Dependence Graph，PDG）を構築した後，変更部分に関係する依存関係を抽出し，サブグラフを構築する．そして，グラフの特徴量をベクトル化した後 GNN に入力，学習を行い，サブグラフ単位で不正な確率を求める．また，同一の変更差分から構築されたサブグラフを集約し，変更単位で不正機能の有無を判定する 2 値分類モデルを構築する．

提案手法では特定の不正機能の実装に限定せず検出対象とする．また，複数の文により構成される依存関係を表現し，学習に用いる．

本研究の貢献を以下に示す．

- ソースコードの変更差分に対する不正機能検出手法を提案した．提案する検出手法では，ソースコードを PDG として表現し，変更差分に関する依存関係を抽出したのち，GNN を用いて変更差分について不正機能の有無を判定する 2 値分類モデルを構築する．
- 提案する不正機能検出手法では，17 件の不正機能を含む更新，および 13 件の正常更新を含むソースコードに対して評価を行い，提案手法が F1 スコア 91.43% を示し，文単位や関数単位で学習する脆弱性検出手法 [3], [4] に比べて分類精度が高いことを確認した．

[1] 神戸大学 大学院工学研究科 電気電子工学専攻
Dept. of Electrical & Electronic Eng., Kobe University
[a)] naoki.yamamoto@gsuite.kobe-u.ac.jp

ソースコード例

```
1  int auth(const char* user, const char* pass) {
2      const char* stored = lookup_password(user);
3      int ok = 0;
4      if (strcmp(pass, stored) == 0)
5          ok = 1;
6      if (ok)
7          grant_access(user);
8      return ok;
9  }
```

ソースコードから構築されるPDG例

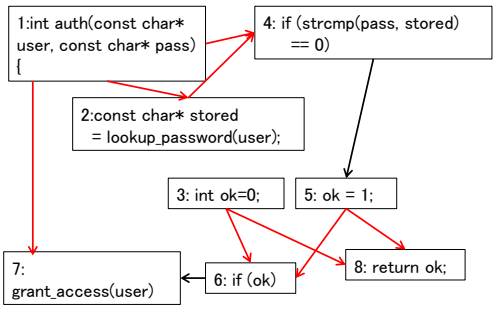


図 1 PDG の構成例：黒色の矢印が制御依存関係，赤色の矢印がデータ依存関係に対応する．

2. 背景知識

2.1 プログラム依存グラフ

プログラム依存グラフとは，プログラム中の文や条件式をノードとし，文間に存在する依存関係をエッジとして表現したグラフのことである．PDG では，以下の 2 種類の依存関係をエッジとして表現する [5]．

- 制御依存関係：プログラムにおいて，ある文の実行可否が他の条件文の判定結果に依存する関係．
- データ依存関係：プログラムにおいて，ある文で定義された変数が他の文で使用される関係．

図 1 に PDG の構成例を示す．上部にソースコード例，下部にソースコード例から構築した PDG を示す．図 1 では，ソースコードの各文をノード，文間の依存関係をエッジとして矢印で表現している．各ノードには対応する文の行番号を付与している．PDG を用いることで制御依存，データ依存関係をともに同一のグラフ表現に含められる．

2.2 グラフニューラルネットワーク

GNN とは，グラフ構造を入力として，ノードやグラフの分類，予測を行う深層学習手法である [3]．各ノードの特

© 2026 Information Processing Society of Japan

表 1 不正機能の分類 [6], [7]：データセット内に分類した不正機能が含まれる場合は ◯，含まれない場合は–と表す．

不正機能	説明	DarkWRT[6]	ROSARUM[7]
認証回避	正規の認証処理をすり抜け，パスワード等なしで保護機能へ到達可能な機能	◯	◯
権限昇格	システムに対する高度なアクセス権を得る機能	◯	◯
設定変更	ユーザの意図していない設定を第三者により変更する機能	◯	◯
任意コード実行	攻撃者が指定したコマンドや処理を実行させる機能	◯	◯
任意メモリ操作	プロセスのメモリ内容を読み出す，または書き換えを行う機能	—	◯
システム破壊，妨害	システムを強制的に利用不能にする機能	◯	◯
情報窃取	ファイルや設定，秘密情報などを不正に読み取り外部に渡す機能	◯	◯
痕跡消去	不正機能の証拠を消去する機能	◯	—

徴量をベクトルとして表し，グラフの接続関係とともに入力する．GNN では，近傍ノードの情報を集約してノード表現を更新するメッセージパッシングという処理を複数層で行い，複数のノードにまたがる構造的な関係を反映した表現を得る [8].

2.3 不正機能

2.3.1 定義

IoT 機器中のファームウェアや OSS 中には設計上のミスや実装上の不備に起因する脆弱性が存在する場合がある．本研究では脆弱性のうち，情報窃取や無断通信，任意コード実行など，「攻撃者が不正に利用可能な特定の機能を持つ脆弱性」を不正機能と定義する．

不正機能に関して既に分類が行われている．既存研究 [6], [7] に基づき，本研究における不正機能を表 1 に分類する．表 1 の分類は，不正機能の挿入により攻撃者が実行可能となる操作に基づく．評価，考察において用いたデータセットの内容を整理するために用いる．

2.3.2 事例

攻撃者が不正機能をソースコードへ挿入する，あるいは開発時の不備により不正機能と同様の機能が残存すると，ソフトウェアの配布を通じて利用者に被害をもたらす．実際に，不正機能を利用した攻撃が発生した事例が報告されている．多くのソフトウェアはサードパーティ製の OSS や商用のコンポーネントに依存しており，一部に不正機能が混入した場合，配布や更新を通じて攻撃に利用される可能性がある．PHP[9] や xz[10] など著名な OSS に対するバックドアの挿入が報告されている [7].

また，IoT 機器のファームウェア中に不正機能が挿入される事例も存在する．ルータのファームウェアにハードコードされた認証情報が存在し，攻撃者が任意のコマンドを実行可能であるという事例が報告されている [11].

3. 関連研究

機械学習を用いた脆弱性検出に関する研究

機械学習を用いて脆弱性検出を行う研究が複数提案されている．LineVD[3] は，C/C++関数内の脆弱な文を特定する文レベル脆弱性検出手法である．関数全体と各文を CodeBERT[12] で埋め込み，PDG を構築する．各文の埋め込みと PDG を GNN に入力して依存関係を考慮した特徴を学習し，関数レベルと文レベルの予測を組み合わせることで，各文の脆弱性を判定する．

LineVul[4] は，CodeBERT を用いて C/C++関数の脆弱性を 2 値分類する検出手法である．関数に脆弱性が含まれるかを判定するとともに，attention 情報に基づいて脆弱性に関係する可能性が高い行を特定する．

不正機能検出に関する研究

不正機能検出を目的として，複数の検出手法が提案されている．Stringer[1] は，ファームウェアに含まれるバイナリを対象とした静的解析手法である．静的データとの比較が制御フローへ与える影響を関数ごとにスコア化し，特定の文字列でのみ到達可能な処理を抽出する．

EAGLEYE[2] は，IoT 機器の利用者向け文書に記載されていない隠し Web インタフェースを探索する手法である．公開インタフェースの要求からルーティングトークンを抽出し，大規模言語モデルで未公開のインタフェースに対応する可能性があるルーティングトークンの候補値を推定し，ブラックボックスファジングを行う．

4. 提案手法

本研究では，GNN を用いて不正機能を検出する手法を提案する．

4.1 要件

本研究で提案する不正機能検出手法は，表 1 にて定義された不正機能のうち，痕跡消去を除く 7 種類の不正機能を対象に静的解析を行う．設計において，次の要件を満たすことを目指した．

- 要件 1：ソースコードの更新前後で挿入される不正機能を検出対象とする．
- 要件 2：ソースコードの更新に伴う変更箇所周辺の制御依存関係，データ依存関係を解析対象に含める．

4.2 設計

4.2.1 設計方針

提案する静的解析手法の設計方針を次のように定めた．

- 設計方針 1：ファームウェアのソースコード，および一部変更し不正機能を挿入したソースコードを用いる．

© 2026 Information Processing Society of Japan

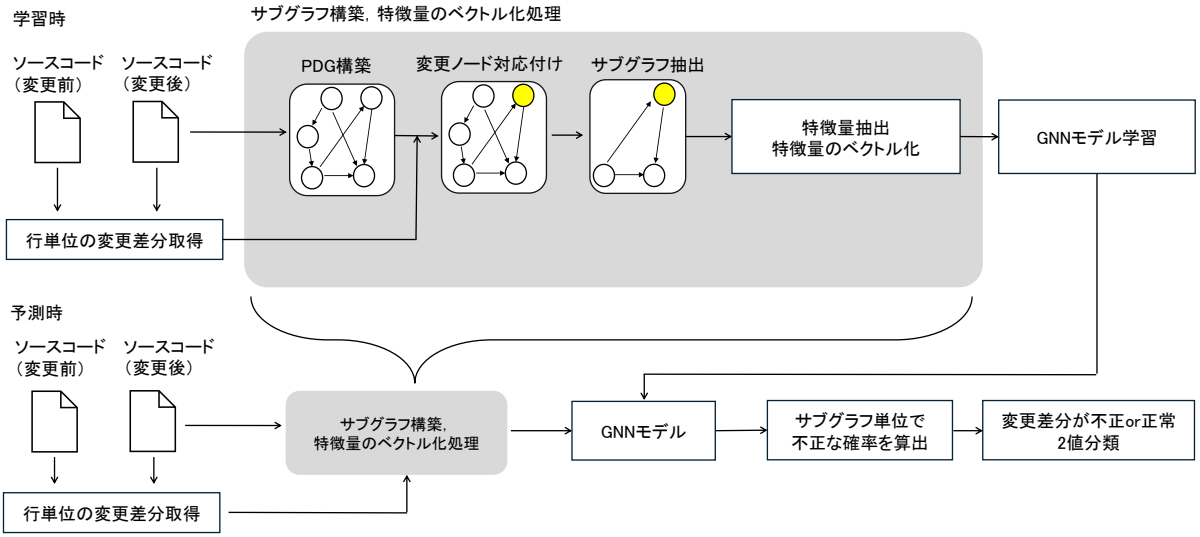


図 2　提案手法の概要：変更差分に対応するノードを黄色で表す．

- 設計方針 2：ソースコードの変更差分を起点として，変更部分に関する依存関係を抽出する．
- 設計方針 3：抽出した依存関係から得られるグラフを用いて GNN による学習を行い，不正機能の有無を判定する 2 値分類モデルを構築する．

4.2.2　設計概要

設計方針に基づく提案手法の概要を図 2 に示す．提案する検出手法では，設計方針に基づき，更新前後のソースコードから差分を取り，更新後ソースコード上の追加行および変更行を取得する．ファイル単位で PDG を構築したのち，変更ノードを中心として周辺の依存関係を抽出し，サブグラフを構築する．そして，サブグラフを GNN へ入力し，サブグラフ単位で不正である確率を算出したのち変更単位で不正機能を含むか判定する．

4.2.3　PDG 構築

変更されたファイルに対し，ソースコードからファイル単位で PDG を有向グラフ $G = (V, E)$ として構築する．ノード集合 V はプログラムの文，条件式などの要素に対応するノードから構成され，エッジ集合 E は文間の制御依存，データ依存関係を表すエッジから構成される．

複数の関数を横断して不正機能が実装される場合を考え，提案手法では更新による変更箇所を含むファイル単位で PDG を構築し，同一ファイル内の複数の行，複数の関数を 1 つのグラフに含める．そのため，本研究では制御依存，データ依存のエッジに加え，関数間の依存関係の表現に必要な関数呼出しの関係，関数呼出しと実引数の関係を示すエッジを追加する．

4.2.4　k-hop サブグラフ構築

k-hop サブグラフを 1 つのノードから距離 k で到達可能なノード，およびノード間のエッジを抽出したグラフと定義する．k-hop サブグラフは以下の手順にて構築する．

- （Step 1）：更新前後のソースコードのテキスト差分を取得し，更新後ファイルにおける追加行，置換行の行番号を変更行番号として記録する．
- （Step 2）：変更行番号と，PDG の各ノードが持つソースコード上の行番号を照合し，変更行番号を持つノードを抽出する．1 行が複数のノードに対応する場合は，ノード集合として扱う．
- （Step 3）：各変更行に対応するノード集合を起点として幅優先探索を行い，PDG 上のノード間距離が k 以下であるノードを抽出し，k-hop サブグラフのノード集合を構築する．また，収集したノード間を接続するエッジを PDG から抽出し，サブグラフのエッジ集合を構築する．

4.2.5　サブグラフの重複判定

各変更行に対してサブグラフを構築すると，ノード，エッジの大部分が重複するサブグラフが複数生じる場合がある．近似した依存関係の学習を抑えるため，Jaccard 係数を計算することで重複したサブグラフの統合を行う．Jaccard 係数は 2 つの集合の類似度を測る指標であり，2 つの集合 A, B の Jaccard 係数 $J(A, B)$ を式 (1) に示す．

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{1}$$

同一ファイルから構築したサブグラフを 2 つ選び，ノード集合，エッジ集合の Jaccard 係数を計算する．2 つの集合の Jaccard 係数が設定した閾値を超える場合 2 つのグラフを重複と判定し，各サブグラフに含まれるノード，エッジを収集して 1 つのサブグラフとして統合，再構築する．

4.2.6　GNN 構築

k-hop サブグラフを入力とし，不正機能を含む変更であるかを判定する 2 値分類モデルを構築する．モデルには，エッジ種別を区別して近傍ノードの特徴を集約可能である Relational Graph Convolutional Network（RGCN）を用いる [13]．各ノードの種別，コードトークン，中心ノード

© 2026 Information Processing Society of Japan

からの距離および特定の API を入力特徴とする．教師ラベルは，不正更新に由来するサブグラフを 1，正常更新に由来するサブグラフを 0 とする．

RGCN の第 l 層におけるノード i の表現更新を式 (2) に示す．

$$\boldsymbol{h}_i^{(l+1)} = \mathrm{ReLU}\left(\boldsymbol{W}_{\mathrm{root}}^{(l)} \boldsymbol{h}_i^{(l)} + \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{\boldsymbol{W}_r^{(l)} \boldsymbol{h}_j^{(l)}}{|\mathcal{N}_i^r|} + \boldsymbol{b}^{(l)} \right) \quad (2)$$

ここで，$\boldsymbol{h}_i^{(l)}$ はノード i の表現，$\mathcal{R}$ は自己ループを含むエッジ種別の集合，$\mathcal{N}_i^r$ は関係 r で接続された近傍ノードの集合である．$\boldsymbol{W}_r^{(l)}$，$\boldsymbol{W}_{\mathrm{root}}^{(l)}$ および $\boldsymbol{b}^{(l)}$ は学習により更新されるパラメータである．

本実装では，各ノードから自身へ自己ループを追加する．自己ループにより，近傍ノードの情報を集約する際にノード自身の特徴も反映し，変更箇所やコード要素が持つ情報を更新後のノード表現に保持する．また，活性化関数には ReLU を用いる [14]．

ノード表現を 2 層の RGCN で更新した後，pooling を適用して固定長ベクトルのグラフ表現を得る．グラフ表現を全結合層に入力して出力値を算出したのち，sigmoid 関数を適用して 0 から 1 の範囲に変換し，サブグラフの不正確率とする．ここで，全結合層の出力値 z_g に対して sigmoid 関数を適用して不正確率 p_g を求める式を式 (3) に示す．そして，同一の変更差分から構築されたサブグラフを集約し，更新による不正確率を全サブグラフの不正確率の最大値として求める．不正確率が設定した閾値より大きければ不正機能を含む更新と判定する．

$$p_g = \frac{1}{1 + \exp(-z_g)} \quad (3)$$

4.3 実装

4.3.1 PDG 構築

PDG 構築には静的解析ツール Joern[15] を用いる．変更前後のファイルに対して変更差分を取り，変更ファイルを Joern へ入力し，joern-parse を用いて中間表現である Code Property Graph（CPG）を生成する．CPG は多くの種類のエッジ種別を含むが，制御依存関係，およびデータ依存関係を示すエッジを抽出し，PDG を構築する．出力形式はグラフの頂点，辺及び属性を JSON 形式で記述する規格である GraphSON[16] を用いる．

本研究では，ROSARUM[7] に含まれる不正更新の依存関係を調査し，多くの不正機能において，外部入力に関係する処理から不正機能を実行する処理までが依存関係上 3 ステップ以内に含まれることを確認した．ステップ数を増やすとより多くの依存関係を表現できるが，不正機能と関係性の低い依存関係を学習する可能性がある．そのため，サブグラフのステップ数は k=3 に設定した．また，Jaccard 係数において，重複度が極めて高いサブグラフを削減するため，閾値を 0.95 と設定し，2 つのサブグラフの Jaccard 係数が 0.95 以上の場合は 1 つのサブグラフへ統合した．

表 2 提案手法の実装，評価に用いたソフトウェア環境

ソフトウェア	バージョン
OS	Debian GNU/Linux 13.4（x86_64）
Apptainer	1.5.0
Java	OpenJDK 21.0.11
Joern	4.0.540
Python	3.11.13
PyTorch	2.7.1+cu128
PyTorch Geometric	2.7.0
CUDA Runtime	12.8

4.3.2 GNN 構築

GNN 構築には PyTorch[17] および PyTorch Geometric[18] を用いる．GNN に入力する特徴量には，ノード種別，コードトークン，中心ノードからの距離，特定の API コマンドを使用する．特徴量をベクトルへ変換した後，2 層の RGCN に入力した．入力特徴ベクトルはノードに関する特徴量の表現を確保しつつ，用いるデータセット規模を考慮して 128 次元に設定した．

5. 評価

5.1 評価項目と目的

既知の不正機能を含むソースコードに対して提案手法を適用し，不正機能の検出精度の検証を目的として評価した．評価項目と内容を以下に示す．

- 解析対象ソースコードの変更に伴う不正機能挿入の検出可否：既知の不正機能を含むソースコードの変更差分から不正機能の有無を分類可能か評価した．
- 解析対象ソースコードの不正機能を含まない変更の誤検出に関する評価：不正機能を含まない正常な更新を含むソースコードを入力し，不正でないと分類可能か評価した．

5.2 評価環境

提案手法の実装および評価に用いた評価用計算機は Intel Xeon 6520P CPU（24 コア，48 スレッド，1 ソケット，メモリ 64GB），および NVIDIA GeForce RTX 5090 GPU（メモリ 32GB）とした．提案手法の実装および評価に用いたソフトウェア環境を表 2 に示す．

5.3 データセット

本研究では，提案手法の実装および評価に以下のデータセットを使用した．

- 正例データセット

  不正機能が挿入された更新を含むデータセットとして ROSARUM[7] を使用した．ROSARUM は，ネットワークルータのファームウェアや OSS への攻撃事例に基づく 7 件の不正機能と，ファジングベンチマー

© 2026 Information Processing Society of Japan

表 3 5-fold 交差検証における各 fold のサブグラフ数：n_0 は正常更新に由来するラベル 0 のサブグラフ数，n_1 は不正更新に由来するラベル 1 のサブグラフ数を表す．

	Train			Valid			Test		
fold	n_0	n_1	Total	n_0	n_1	Total	n_0	n_1	Total
0	79	134	213	3	5	8	31	15	46
1	84	127	211	5	7	12	24	20	44
2	87	108	195	3	11	14	23	35	58
3	81	105	186	12	10	22	20	39	59
4	80	100	180	18	9	27	15	45	60

ク MAGMA を基に研究用に作成された 10 件の不正機能の計 17 件から構成される．各サンプルソフトウェア（以下，サンプル）について，正常版と不正機能を含む不正版が含まれる．

本研究では ROSARUM に含まれる 17 種類のサンプルを正例データセットとして使用した．各サンプルについて，正常版を変更前，不正版を変更後とした．

- 負例データセット

  ROSARUM に収録された 17 件のサンプルのうち，対象ソフトウェアの公式リポジトリまたは公式配布物から更新前後のソースコードを取得できた 13 件を対象とし，各対象から 1 件ずつ，合計 13 件のソースコードの正常な更新を収集した．正常な更新には，通常の開発過程で行われた機能追加，バグ修正，および脆弱性修正等が含まれる．

本研究ではデータセットから構築した k-hop サブグラフを用いて学習，評価を実施した．評価では K-分割交差検証 [19] を用いた．K-分割交差検証では全サンプルが一度ずつ評価用データとして用いられ，単一分割に比べて学習，評価の分割方法に関する影響を軽減した評価が可能である．

評価において，K-分割交差検証を K=5 に設定した．すべてのデータセットを 5 個のグループ（fold）に分割し，各 fold を一度ずつ Test に割り当てた．残りの fold を Train および Valid に分割した．Train を用いてモデルを学習し，Valid に基づいて評価対象のモデルを選択した．また，5 回の試行に関して，モデルの重みの初期化に用いる乱数シードはすべて 1 に固定した．

評価の分割に関して，正常/不正で類似したサンプルから得られたサブグラフが Train と Test の両方に混在すると固有の特徴が学習され，過大評価される可能性がある．固有の特徴の学習を防ぐために，対応する同一システム由来のサンプルは同一グループとして扱うよう注意した．

各 fold の Train，Valid および Test に含まれるサブグラフ数を表 3 に示す．表中の n_0 は正常更新に由来するラベル 0 のサブグラフ数，n_1 は不正更新に由来するラベル 1 のサブグラフ数を表す．

5.4 評価結果

5.4.1 提案手法の分類結果

同一の更新に属するサブグラフの不正確率の最大値を更新スコアとし，更新スコアが 0.5 以上の場合に不正更新と判定した．各 fold の Test に対する分類結果を表 4 に示す．提案手法が不正機能を含む更新 17 件中 16 件を検出，正常更新 13 件中 11 件を正常と分類したことを確認した．

表 4 各 fold における提案手法の分類結果

fold	Accuracy	Precision	Recall	F1	TP	FP	TN	FN
0	50.00%	50.00%	50.00%	50.00%	1	1	1	1
1	100.00%	100.00%	100.00%	100.00%	4	0	3	0
2	80.00%	75.00%	100.00%	85.71%	3	1	1	0
3	100.00%	100.00%	100.00%	100.00%	4	0	3	0
4	100.00%	100.00%	100.00%	100.00%	4	0	3	0
全体	90.00%	88.89%	94.12%	91.43%	16	2	11	1

5.4.2 不正機能を含む変更の検出可否に関する評価

提案手法の分類性能を確認するため，比較手法として機械学習を用いた脆弱性検出手法である LineVD[3] と LineVul[4] を用いた．比較評価を行う際，提案手法と同一の不正更新 17 件を用い，共通の 5-fold 交差検証と乱数シードを適用した．また，各手法の出力を更新単位に集約し，不正/正常と分類した．

不正更新 17 件に対する手法別の分類結果を表 5 に示す．各サンプルについて，表 1 に基づく不正機能の分類，および文間依存の有無を示す．各不正更新について，複数の文に横断して不正機能の処理が含まれる場合，文間依存が存在するとした．LineVD は不正機能を含む更新 17 件中 10 件，LineVul は 17 件中 7 件を検出した．

5.4.3 不正機能を含まない変更の誤検出に関する評価

提案手法と同一の正常更新 13 件を使用し，各手法の出力を更新単位に集約し，正常または不正に分類した．手法別の分類結果を表 6 に示す．正常更新 13 件中，LineVD は 10 件，LineVul は 5 件を正しく分類した．

6. 考察

6.1 評価に対する考察

表 4 より，提案手法の F1 スコアは 91.43%を示し，不正機能を含む更新，および正常な更新について高い精度を示した．また，表 5 より，提案手法が脆弱性検出手法と比べ，より高い不正機能検出精度を示すことを確認した．

提案手法では `php-8.0.20` の不正機能のみ検出に失敗したが，`php-8.0.20` は他の脆弱性検出手法においても検出に失敗した．原因として，`php-8.0.20` では特定の入力時に `zend_eval_string` により文字列を PHP コードとして評価，実行を行う処理を追加している．データセット内の他の不正機能と実行形態が異なり，不正を示す特徴だと学習できず，検出に失敗したと考えられる．

4 件の不正機能を含むサンプル[*1]に対し，2 つの脆弱性検出手法で検出に失敗し，提案手法で検出に成功した．例として `libtiff-4.3.0` について，LineVD は文単位で不正，関数単位では正常と判定したが，出力は正常判定となり誤

[*1] `libtiff-4.3.0`，`openssl-3.0.0`，`poppler-21.07.0`，および `sudo-1.9.15p5`

© 2026 Information Processing Society of Japan

表 5 不正更新 17 件に対する文間依存の有無と手法別の分類結果：サンプルが文間依存を含む場合 ✓ と表示する．◯ は不正な更新と正しく分類，× は誤分類を表す．

サンプル	不正機能	文間依存	提案手法	LineVD	LineVul
belkin-f9k1102-httpd	任意コード実行	✓	◯	◯	◯
d-link-1.13A-thttpd	認証回避	✓	◯	×	◯
libpng-1.6.43	任意コード実行	✓	◯	◯	×
libsndfile-1.2.2	システム破壊，妨害	✓	◯	◯	×
libtiff-4.3.0	任意コード実行	✓	◯	×	×
libxml2-2.9.12	任意コード実行	✓	◯	◯	◯
linksys-openwag200g-scfgmgr	設定変更，情報窃取，任意メモリ操作	✓	◯	◯	×
lua-5.4.7	情報窃取	✓	◯	◯	◯
openssl-3.0.0	任意コード実行	✓	◯	×	×
php-8.0.20	任意コード実行	✓	×	×	×
php-8.1.0-dev	任意コード実行	✓	◯	◯	×
poppler-21.07.0	任意コード実行	✓	◯	×	×
proftpd-1.3.3c	任意コード実行	✓	◯	◯	◯
sqlite3-3.37.0	システム破壊，妨害	✓	◯	◯	×
sudo-1.9.15p5	認証回避，権限昇格	✓	◯	×	×
tenda-w302r-httpd	任意コード実行	✓	◯	×	◯
vsftpd-2.3.4	任意コード実行	✓	◯	◯	◯
検出数		–	16/17	10/17	7/17

分類とした．LineVul は検出対象関数が大きく入力トークン数の上限に達し，不正機能に関連する処理を直接評価できず，出力は正常判定となり誤分類とした．

比較した脆弱性検出手法に関して，LineVD は行単位の脆弱性検出を目的とする．不正機能が外部入力の取得から危険な処理の実行まで複数の文，行を横断して実装される場合，各文の予測を更新単位へ集約しても処理間の関係を十分に反映できず検出精度が低いと考えられる．また，関数単位の判定も行うため，関数が大きい場合に判定を誤る場合がある．LineVul は関数単位の脆弱性検出を目的とするため，入力する関数が大きい場合，不正機能に関係する依存関係を十分に学習できず検出精度が低いと考えられる．提案手法では脆弱性検出手法と比較して，変更部分に対して複数の文間を横断する依存関係を抽出し学習に用いるため，より多くの不正機能を検出可能だと考えられる．

6.2 提案手法に対する考察

6.2.1 提案手法の設計，実装

本研究の提案手法では，ソースコードの更新により挿入される不正機能を検出対象とし，ソースコードの変更前後の依存関係を抽出して GNN の学習に用いる．提案手法では検出対象として更新前と更新後のソースコードを必要とするが，変更差分を取得することで変更部分に関する依存関係を抽出し，不正機能の依存関係，正常な更新の依存関係について学習可能とした．

また，本研究では，GNN モデルにより出力された不正な確率を集約し，設定した閾値 0.5 を超える場合に不正と判断した．閾値について，LineVD ではデータセットの不均衡を考慮し，Valid における F1 値が最大となる閾値を選択した上で，Test を評価する [3]．本研究では Valid のデータ数が少なく，fold により閾値が不安定になることを防ぐため，全 fold で閾値を固定した．評価結果として，閾値 0.5 でも正常/不正な更新を分類可能であると確認した．

表 6 正常更新 13 件に対する手法別の分類結果：◯ は正常更新と正しく分類，× は誤分類を表す．

システム	提案手法	LineVD	LineVul
libpng-1.6.43	◯	◯	◯
libsndfile-1.2.2	◯	◯	◯
libtiff-4.3.0	×	◯	×
libxml2-2.9.12	◯	×	×
lua-5.4.7	◯	◯	×
openssl-3.0.0	◯	◯	◯
php-8.0.20	◯	◯	◯
php-8.1.0-dev	◯	◯	×
poppler-21.07.0	◯	◯	×
proftpd-1.3.3c	◯	◯	×
sqlite3-3.37.0	◯	×	×
sudo-1.9.15p5	◯	×	◯
vsftpd-2.3.4	×	◯	×
正解数	11/13	10/13	5/13

6.2.2 関連研究との比較

本研究と関連研究 [1], [2], [3], [4] との比較を表 7 に示す．提案手法では，ファイル単位で PDG を構築するため，不正機能が複数の変更行に横断して実装される場合でも，1 つの変更箇所で得られた不正機能の特徴を表現可能である．一方で，LineVD は文単位，LineVul は関数単位を主な分類単位とし，不正機能を構成する処理が複数の文や関数に分けられる場合，検出が難しいと考えられる．

不正機能検出に関して，Stringer は静的データとの比較，EAGLEYE は隠し Web インタフェースを主な解析対象とするのに対し，提案手法では検出対象を限定せず，学習を用いて不正機能の依存関係を解析対象とする．しかし，提案手法では学習データに含まれる不正機能と依存関係が大きく異なる不正機能について，不正を示す特徴を十分に捉えられず，検出性能が低下する可能性がある．

6.2.3 限界

本研究では，ROSARUM に収録された 17 サンプルの不正更新を正例として使用し，正常な更新を負例として収集した．しかし，正常更新を収集できたのは 13 システムであり，4 システム[*2]は収集を断念した．該当サンプルの不正機能は，ベンダが独自に実装または変更した `httpd`，`thttpd`，`scfgmgr` 等のプログラムに含まれるため，対象製品に対応するソースコードの更新を取得できていない．

また，提案手法ではファイル内のプログラムの依存関係をグラフとして表現したが，ファイル間を横断する依存関係を表現していない．PDG 構築に用いた Joern では複数のファイルから CPG を構築可能だが，解析するノード数の増加に伴い CPG 構築に失敗する場合が存在する．そのため，関数間を横断する依存関係が実装されている場合，検出に必要な依存関係が不足すると考えられる．

[*2] `belkin-f9k1102-httpd`，`d-link-1.13A-thttpd`，`linksys-openwag200g-scfgmgr`，および `tenda-w302r-httpd`

© 2026 Information Processing Society of Japan

表 7　提案手法と既存研究の比較

手法	判定単位	検出対象	入力	制約
提案手法	更新単位	不正機能を含む更新	ソースコード	更新前後のソースコードを必要とする.
LineVD[3]	関数単位, 文単位	脆弱性を含む文	ソースコード	複数の文を横断する依存関係の表現が難しい.
LineVul[4]	関数単位	脆弱性を含む関数	ソースコード	関数サイズが大きい場合, 不正機能に関する依存関係の検出が難しい.
Stringer[1]	関数単位	静的データ比較に基づく不正機能	バイナリ	静的データとの比較を伴わない不正機能への適用が難しい.
EAGLEYE[2]	Web インタフェース単位	隠し Web インタフェース	ファームウェアへの入力要求	Web インタフェースを介さない不正機能への適用が難しい.

7. おわりに

本研究では，ファームウェア更新時に挿入される不正機能に対し，GNN を用いた検出手法を提案した．提案する不正機能検出手法では，ソースコードから PDG を構築し，変更差分に関する依存関係を抽出後，GNN を用いて変更差分の不正機能有無を判定する 2 値分類モデルを構築する．

提案する不正機能検出手法を，不正機能データセットである ROSARUM，および正常な更新を含むソースコードに対して適用し，不正機能の有無を分類可能か，また正常な更新を分類可能か評価した．評価結果において，提案手法が F1 スコア 91.43%を示し，高い分類精度を持つと確認した．また，関連研究である脆弱性検出手法と比較して不正機能の検出件数が多いこと，誤検出が少ないことを確認した．課題として，正常更新データセットの不足，ファイル間の依存関係の不足が挙げられる．

謝辞　本研究の一部は，JSPS 科研費 JP26K21217，ならびに JST 経済安全保障重要技術育成プログラム JPMJKP24K2 の支援を受けたものである．

参考文献

[1] Thomas, S.L., Chothia, T. and Garcia, F.D.: Stringer: Measuring the Importance of Static Data Comparisons to Detect Backdoors and Undocumented Functionality, Proc. *22nd European Symposium on Research in Computer Security*, Lecture Notes in Computer Science, Vol.10493, pp.513–531, Springer (2017).

[2] Liu, H., Zheng, L., Gan, S., et al.: EAGLEYE: Exposing Hidden Web Interfaces in IoT Devices via Routing Analysis, Proc. *Network and Distributed System Security Symposium*, pp.1–18, (2025).

[3] Hin, D., Kan, A., Chen, H., et al.: LineVD: Statement-level Vulnerability Detection Using Graph Neural Networks, Proc. *19th IEEE/ACM International Conference on Mining Software Repositories*, pp.596–607, (2022).

[4] Fu, M. and Tantithamthavorn, C.: LineVul: A Transformer-based Line-level Vulnerability Prediction, Proc. *19th IEEE/ACM International Conference on Mining Software Repositories*, pp.608–620, (2022).

[5] 兼光智子，肥後芳樹，楠本真二：プログラム依存グラフを用いたリファクタリング候補の特定と可視化，電子情報通信学会技術研究報告，Vol.110，No.336，pp.61–66 (2010)．

[6] Uezono, D., Kushibiki, J., Sasaki, T., et al.: Poster: DarkWrt: Towards Building a Dataset of Potentially Unwanted Functions in IoT Devices, Proc. *Network and Distributed System Security Symposium*, pp.1–2, (2025).

[7] Kokkonis, D., Marcozzi, M., Decoux, E., et al.: ROSA: Finding Backdoors with Fuzzing, Proc. *47th IEEE/ACM International Conference on Software Engineering*, pp.2816–2828, (2025).

[8] Gilmer, J., Schoenholz, S.S., Riley, P.F., et al.: Neural Message Passing for Quantum Chemistry, Proc. *34th International Conference on Machine Learning*, Vol.70, pp.1263–1272, (2017).

[9] Popov, N.: Changes to Git Commit Workflow, PHP Internals Mailing List (online), available from 〈https://news-web.php.net/php.internals/113838〉 (accessed 2026-02-01).

[10] Freund, A.: Backdoor in Upstream xz/liblzma Leading to SSH Server Compromise, oss-security Mailing List (Openwall) (online), available from 〈https://www.openwall.com/lists/oss-security/2024/03/29/4〉 (accessed 2026-02-01).

[11] National Institute of Standards and Technology: CVE-2025-34117 Detail, National Vulnerability Database (online), available from 〈https://nvd.nist.gov/vuln/detail/CVE-2025-34117〉 (accessed 2026-01-29).

[12] Feng, Z., Guo, D., Tang, D., et al.:CodeBERT: A Pre-Trained Model for Programming and Natural Languages,Proc. *Findings of EMNLP 2020*, pp.1536–1547 (2020).

[13] Schlichtkrull, M., Kipf, T.N., Bloem, P., et al.: Modeling Relational Data with Graph Convolutional Networks, Proc. *15th International Conference on The Semantic Web*, Lecture Notes in Computer Science, Vol.10843, pp.593–607, Springer (2018).

[14] Nair, V. and Hinton, G.E.: Rectified Linear Units Improve Restricted Boltzmann Machines, Proc. *27th International Conference on Machine Learning*, pp.807–814, (2010).

[15] Joern: Joern – The Bug Hunter's Workbench (online), available from 〈https://joern.io/〉 (accessed 2026-02-01).

[16] Apache TinkerPop: IO Reference: GraphSON (online), available from 〈https://tinkerpop.apache.org/docs/3.8.0/dev/io/〉 (accessed 2026-02-01).

[17] Paszke, A., Gross, S., Massa, F., et al.: PyTorch: An Imperative Style, High-Performance Deep Learning Library, Proc. *33rd Conference on Neural Information Processing Systems*, Vol.32, pp.8024–8035, (2019).

[18] Fey, M. and Lenssen, J.E.: Fast Graph Representation Learning with PyTorch Geometric, Proc. *ICLR Workshop on Representation Learning on Graphs and Manifolds* (2019).

[19] Bengio, Y. and Grandvalet, Y.: No Unbiased Estimator of the Variance of K-Fold Cross-Validation, *J. Mach. Learn. Res.*, Vol.5, pp.1089–1105 (2004).

© 2026 Information Processing Society of Japan